

JARINGAN KOMPUTER - MODUL 8

TOPIK TUGAS BESAR JARINGAN KOMPUTER

Implementasi dan Analisis Kinerja Sistem Client–Proxy–Server Berbasis Socket Python:
Evaluasi Protokol TCP/UDP dan Parameter Quality of Service

UNTUK MEMBANTU TUGAS BESAR BERIKUT DIBERIKAN LINK
MATERI BERKAITAN: [SOCKET](#), [WEBSERVER](#), [TCP](#), [UDP](#), dan [ICMP](#)

A. Latar Belakang

Perkembangan jaringan komputer modern menuntut sistem komunikasi yang andal, efisien, dan terukur secara kuantitatif. Dalam praktiknya, arsitektur jaringan jarang bersifat langsung antara *client* dan *server*, melainkan sering menggunakan komponen perantara (*intermediary*) untuk meningkatkan kontrol, keamanan, dan performa.

Salah satu arsitektur yang banyak diadopsi adalah model **Client–Proxy–Server**, dengan *proxy server* sebagai penghubung antara *client* dan *server*. Arsitektur ini digambarkan pada Gambar 1.

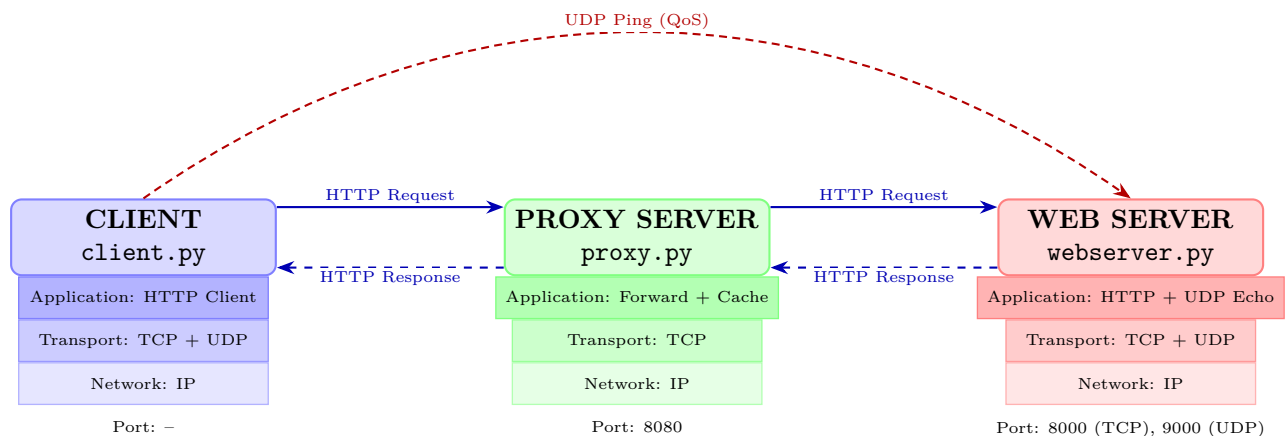


Figure 1: Arsitektur sistem Client–Proxy–Server dengan representasi *protocol stack*

Arsitektur ini banyak diterapkan pada sistem nyata seperti *Content Delivery Network* (CDN) dan jaringan *enterprise* karena mampu menyediakan:

- *Centralized control* untuk manajemen kebijakan jaringan,
- Optimasi performa melalui mekanisme *caching* pada proxy,
- Peningkatan keamanan dengan menyembunyikan topologi server dari akses langsung *client*.

Pemahaman arsitektur ini memerlukan implementasi komunikasi jaringan berbasis *socket programming* yang memanfaatkan dua protokol utama dengan karakteristik berbeda:

- **TCP** (*Transmission Control Protocol*): *connection-oriented*, andal, dan menjamin urutan paket.
- **UDP** (*User Datagram Protocol*): *connectionless*, ringan, dan cocok untuk pengujian performa atau aplikasi *real-time*.

Untuk memastikan keseragaman implementasi, tugas ini menetapkan ketentuan wajib berikut:

1. Struktur File: Sistem harus diimplementasikan menggunakan **tepat tiga file Python**:

- `client.py` – Mengintegrasikan HTTP *client* dan UDP *pinger*.
- `proxy.py` – Mengimplementasikan mekanisme *forwarding* dan *caching*.
- `webserver.py` – Menggabungkan HTTP *server* (TCP) dan UDP *echo server*.

2. Integrasi Kode: File Python tambahan **tidak diperbolehkan**; seluruh logika harus terintegrasi dalam ketiga file tersebut.

3. Asset Pendukung: File HTML beserta seluruh resource pendukung **sudah disediakan** dan tidak boleh dimodifikasi strukturnya.

Melalui tugas ini, mahasiswa diharapkan mampu:

- ▶ Memahami komunikasi jaringan *end-to-end* berbasis *socket*,
- ▶ Menganalisis perbedaan karakteristik dan kasus penggunaan TCP versus UDP,
- ▶ Mengimplementasikan protokol HTTP secara praktis,
- ▶ Merancang peran *proxy server* dalam *forwarding* dan *caching*,
- ▶ Melakukan analisis *Quality of Service* (QoS) sederhana,
- ▶ Memahami komunikasi *multi-hop* dalam topologi jaringan.

Pendekatan ini dirancang agar mahasiswa tidak hanya menguasai teori, tetapi juga mampu merancang dan mengimplementasikan sistem jaringan yang mendekati skenario *real-world deployment*.

B. Tujuan

Tugas besar ini bertujuan memberikan pemahaman mendalam tentang implementasi sistem jaringan berbasis arsitektur *Client-Proxy-Server* serta melatih kemampuan analisis performa jaringan melalui pendekatan praktis dan terukur.

Secara spesifik, tujuan pembelajaran dirumuskan sebagai berikut:

1. Memahami komunikasi jaringan berbasis *socket*

Mahasiswa mampu memahami prinsip dasar *socket programming* dan mengimplementasikan komunikasi jaringan menggunakan:

- Protokol **TCP** (*connection-oriented, reliable*)
- Protokol **UDP** (*connectionless, lightweight*)

2. Mengimplementasikan *Web Server* berbasis TCP

Mahasiswa mampu membangun *web server* sederhana yang dapat:

- Menerima dan memproses HTTP *request* (minimal metode **SYNGET**)
- Melakukan parsing terhadap header dan body request
- Mengirimkan HTTP *response* dengan status code yang sesuai
- Menangani error umum seperti **404 Not Found** dan **500 Internal Server Error**

3. Mengimplementasikan *Proxy Server*

Mahasiswa mampu membangun *proxy server* yang berfungsi sebagai perantara dengan kemampuan:

- Menerima dan memvalidasi request dari *client*

- Meneruskan (*forward*) request ke *web server* tujuan
- Mengembalikan response dari server ke *client*
- Mengimplementasikan mekanisme *caching* sederhana untuk optimasi performa

4. Mengembangkan modul pengujian performa berbasis UDP

Mahasiswa mampu membuat modul pengujian jaringan yang dapat:

- Mengirim paket UDP secara periodik dengan payload terkontrol
- Mengukur *Round Trip Time* (RTT) untuk setiap paket
- Menghitung persentase *packet loss* selama sesi pengujian
- Menghasilkan statistik performa (min/avg/max RTT dan jitter)

5. Mengintegrasikan seluruh komponen sistem

Mahasiswa mampu menyatukan ketiga komponen dalam satu sistem utuh dengan alur komunikasi:



dengan ketentuan **tanpa komunikasi langsung** antara *client* dan *web server*.

6. Menganalisis *Quality of Service* (QoS)

Mahasiswa mampu mengukur dan mengevaluasi performa jaringan berdasarkan parameter:

- **Throughput:** Laju transfer data efektif (bps)
- **Latency:** Waktu tempuh paket (RTT dalam ms)
- **Packet Loss:** Persentase paket yang hilang (%)
- **Jitter:** Variasi delay antar-paket (ms)

7. Mengasah kemampuan *problem solving* pada sistem jaringan

Mahasiswa mampu mengidentifikasi, mendiagnosis, dan menyelesaikan permasalahan implementasi, seperti:

- *Timeout* dan kegagalan koneksi
- Kesalahan parsing HTTP header/body
- Sinkronisasi data pada mekanisme *caching*
- Penanganan error pada protokol UDP yang bersifat *unreliable*

Capaian Kompetensi: Pencapaian tujuan di atas membekali mahasiswa dengan kemampuan teoritis dan praktis untuk merancang, mengimplementasikan, dan menganalisis sistem jaringan komputer secara menyeluruh, mencakup pemahaman mendalam terhadap:

- ▶ **Application Layer:** Protokol HTTP, arsitektur *web server*, dan mekanisme *proxy server*.
- ▶ **Transport Layer:** Komunikasi andal TCP versus komunikasi ringan UDP, serta implikasinya terhadap desain aplikasi.
- ▶ **Network Layer:** Konsep dasar IP routing dan referensi diagnostik jaringan (*traceroute*, ICMP).
- ▶ **Cross-Layer Analysis:** Evaluasi parameter QoS dari perspektif multi-layer untuk optimasi sistem.

C. *Tools* dan Teknologi

Tugas besar ini memanfaatkan sejumlah *tools* dan teknologi untuk mendukung proses implementasi, pengujian, dan analisis sistem jaringan. Komponen tersebut diklasifikasikan menjadi persyaratan wajib dan pendukung:

1. Bahasa Pemrograman Python

Seluruh implementasi wajib menggunakan Python 3. Bahasa ini dipilih karena menyediakan modul `socket` bawaan yang mendukung komunikasi jaringan berbasis TCP dan UDP secara langsung.

2. *Socket Programming* (TCP & UDP)

Komunikasi jaringan harus diimplementasikan pada tingkat *low-level* menggunakan modul `socket`, tanpa bantuan *framework* atau pustaka tingkat tinggi.

- **TCP:** Digunakan untuk komunikasi HTTP antara *client*, *proxy*, dan *web server*.
- **UDP:** Digunakan khusus untuk pengujian performa jaringan (QoS *ping/echo*).

3. Sistem Operasi

Lingkungan eksekusi dapat menggunakan **Linux**, **macOS**, atau **Windows**. Pemilihan sistem operasi **disesuaikan** dengan ketersediaan dan preferensi mahasiswa, selama mendukung Python 3 dan pemrograman *socket*.

4. *Text Editor* atau *IDE*

Digunakan untuk pengembangan kode, seperti Visual Studio Code, PyCharm, atau Thonny. Disarankan menggunakan *environment* yang mendukung *syntax highlighting* dan fitur *debugging* Python.

5. *Web Browser*

Digunakan untuk verifikasi manual respons HTTP (opsional), seperti Google Chrome atau Mozilla Firefox. *Catatan:* Browser hanya berfungsi sebagai alat verifikasi, bukan pengganti implementasi `client.py`.

6. Wireshark

Packet analyzer wajib digunakan untuk menangkap dan menganalisis lalu lintas paket selama pengujian. Wireshark memfasilitasi analisis QoS secara visual, termasuk verifikasi *handshake* TCP, struktur paket UDP, serta identifikasi *retransmission* dan *packet loss*.

7. Jaringan Lokal (*LAN*)

Seluruh komponen (*client*, *proxy*, *server*) harus beroperasi dalam satu segmen jaringan yang sama (LAN, WiFi, atau *hotspot* lokal) untuk memastikan konektivitas stabil dan pengukuran QoS yang akurat tanpa interferensi *routing* eksternal.

Catatan Penting & Batasan Implementasi

- ▶ Penggunaan *framework* web (Flask, Django, FastAPI) atau pustaka HTTP tingkat tinggi (`requests`, `http.server`, dll.) **tidak diperbolehkan**.
- ▶ Seluruh mekanisme komunikasi jaringan wajib diimplementasikan secara manual menggunakan modul `socket`.

D. Topologi dan Pembagian Peran

Tugas besar ini mengimplementasikan arsitektur jaringan berbasis model *Client-Proxy-Server*. Seluruh komunikasi antara *client* dan *web server* wajib melewati *proxy server* sebagai perantara terpusat.

1. Topologi Sistem

Topologi komunikasi mengikuti alur berikut:

Client $\xrightarrow{\text{TCP/UDP}}$ Proxy Server $\xrightarrow{\text{TCP/UDP}}$ Web Server

Ketentuan wajib:

- ▶ *Client* **tidak diperbolehkan** berkomunikasi langsung dengan *Web Server* melalui TCP (HTTP).
- ▶ Seluruh HTTP *request* wajib dialirkan melalui *Proxy Server*.
- ▶ *Proxy Server* menjadi satu-satunya titik transit antara *client* dan *web server*.

Relevansi Praktis: Arsitektur ini merepresentasikan implementasi nyata pada jaringan *enterprise* dan CDN, yang memungkinkan:

- **Centralized control:** Monitoring, filtering, dan modifikasi *traffic* pada satu titik.
- **Optimasi performa:** *Caching* mengurangi beban server dan *latency*.
- **Peningkatan keamanan:** Isolasi *web server* dari akses langsung *client*.

2. Alur Komunikasi Sistem

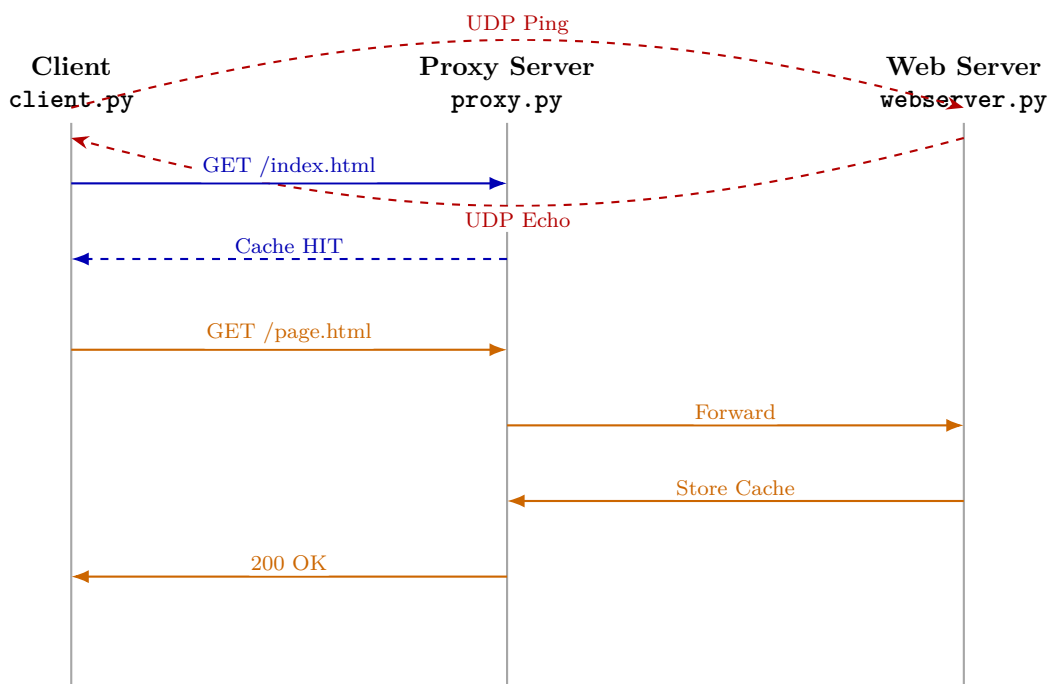


Figure 2: Sequence diagram alur komunikasi TCP/UDP dan mekanisme cache

Keterangan Alur:

- **TCP Cache HIT:** Proxy merespons langsung dari penyimpanan lokal tanpa menghubungi server.
- **TCP Cache MISS:** Proxy meneruskan *request* ke server, menyimpan respons, lalu mengirimkannya ke *client*.
- **UDP QoS:** Koneksi *connectionless* untuk pengukuran RTT, *jitter*, dan *packet loss*.

3. Pembagian Peran Berdasarkan Jumlah Anggota

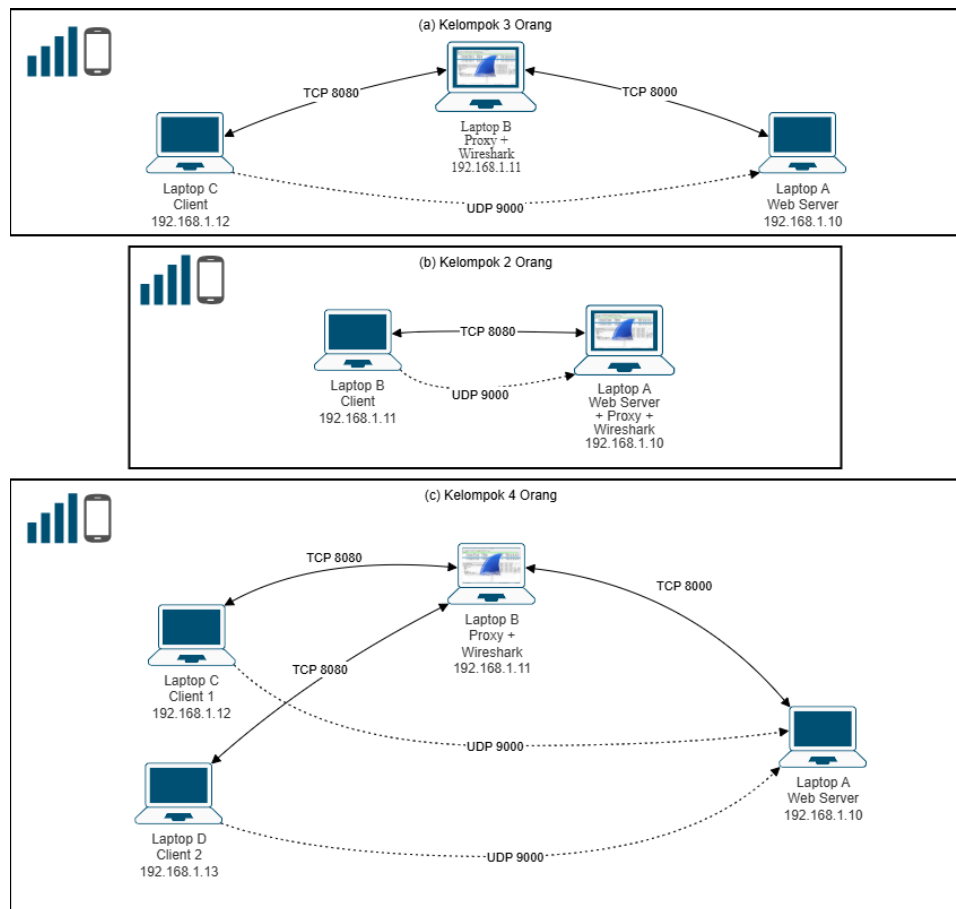


Figure 3: Skenario Kelompok untuk Anggota 3 (**utama**) / 2 atau 4 (khusus)

Catatan Khusus Implementasi Per Kelompok

Kelompok 2 Orang: *Web Server* dan *Proxy* dijalankan pada perangkat yang sama (Laptop A), namun implementasi **tetap wajib menggunakan 3 file Python terpisah**. Keduanya berjalan sebagai proses independen dengan port berbeda (*co-located*).

Kelompok 4 Orang: Peran *Client* dibagi menjadi dua instance dengan skenario beban berbeda. Keduanya mengimplementasikan seluruh fitur yang diminta, namun berbeda pada mekanisme pengiriman *request*:

- **Client 1:** Menggunakan mekanisme *single request* (satu permintaan per waktu).
- **Client 2:** Menggunakan mekanisme *multi-request* atau koneksi konkuren (beberapa permintaan bersamaan).

Kedua *client* berkomunikasi melalui *Proxy* yang sama untuk mensimulasikan trafik yang lebih realistis.

4. Peran Setiap Komponen

- ▶ **Client (`client.py`):** Menginisiasi HTTP *request* ke Proxy, menampilkan respons, dan menjalankan modul UDP *pinger* untuk pengujian QoS.
- ▶ **Proxy Server (`proxy.py`):** Bertindak sebagai *gateway*; menerima *request*, memeriksa *cache*, meneruskan ke server (jika *miss*), menyimpan respons, dan mengembalikan data ke *client*.
- ▶ **Web Server (`webserver.py`):** Menerima *request* dari Proxy, menyajikan file statis, menyusun HTTP *response*, menangani error (404, 500), dan menyediakan layanan UDP *echo server*.

5. Ketentuan Implementasi Topologi

- ▶ Seluruh perangkat wajib terhubung dalam satu jaringan lokal (LAN/WiFi/*Hotspot*) tanpa NAT routing eksternal.
- ▶ Konfigurasi IP dan port harus konsisten serta diverifikasi sebelum pengujian.
- ▶ Sistem operasi yang digunakan dibebaskan, namun disarankan mendukung **socket API** secara *native*.

Pertimbangan Teknis:

- **Kompatibilitas lintas-platform:** Perhatikan perbedaan *line ending* (CRLF vs LF), pemisah jalur (*path separator*), dan konfigurasi *firewall* bawaan OS.
- **Sinkronisasi waktu:** Akurasi pengukuran RTT dan *jitter* bergantung pada kesamaan waktu antar perangkat. Disarankan menggunakan NTP atau sinkronisasi manual sebelum pengujian UDP.
- **Stabilitas jaringan:** Gunakan IP statis atau *DHCP reservation* untuk mencegah perubahan alamat yang dapat memutus koneksi *socket* selama pengujian.

E. Ketentuan Implementasi

Bagian ini memuat spesifikasi teknis yang bersifat **wajib**. Seluruh implementasi harus mematuhi ketentuan berikut tanpa pengecualian.

1. Ketentuan Umum

Aturan Dasar:

- ▶ **Bahasa:** Python 3.x (tanpa *framework* web seperti Flask, Django, atau FastAPI).
- ▶ **Komunikasi:** Implementasi komunikasi wajib menggunakan modul `socket` secara manual (`socket.socket`).
- ▶ **Struktur File:** Sistem harus terdiri dari **tepat tiga file Python**:

`client.py   ·   proxy.py   ·   webserver.py`
- ▶ **Larangan:** Pembuatan file tambahan (helper, modul terpisah, `udp.py`, dll.) **tidak diperbolehkan**.
- ▶ **Aset:** Berkas HTML dan resource pendukung **sudah disediakan dan tidak boleh dimodifikasi**. Letakkan pada direktori yang sama dengan `webserver.py`.

2. Spesifikasi Komponen

a. Web Server (`webserver.py`)

• TCP Server (HTTP):

- Menggunakan `SOCK_STREAM` dan terikat pada port 8000 (dapat diubah jika port sedang digunakan).
- Menerima dan mengurai (*parsing*) permintaan HTTP GET.
- Mengirim *response* dengan format HTTP/1.1 yang valid:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: <size>

<file_content>
```

- Menangani galat standar: 404 Not Found (berkas tidak ditemukan) dan 500 Internal Server Error.
- Mencatat log: IP *client*, jalur berkas, *timestamp*, dan *status code*.

• UDP Server (QoS Echo):

- Menggunakan `SOCK_DGRAM` pada port 9000 (dapat diubah jika port sedang digunakan).
- Menerima dan memantulkan (*echo*) paket UDP tanpa mengubah *payload*.

- **Konkurensi:** Wajib menangani beberapa koneksi simultan menggunakan `threading` atau `select()`.

b. Proxy Server (`proxy.py`)

- Menggunakan `SOCK_STREAM` pada port 8080 (dapat diubah jika port sedang digunakan).
- **Forwarding:** Menerima *request* dari *client*, meneruskan ke *Web Server*, dan mengembalikan *response*.
- **Caching (WAJIB):**

- ◊ Menyimpan *response* HTTP ke berkas lokal berdasarkan jalur URL.
- ◊ *Cache HIT*: Mengirim *response* langsung dari berkas cache tanpa menghubungi server.
- ◊ *Cache MISS*: Meneruskan *request* ke server, menyimpan *response*, lalu mengirimkannya ke *client*.
- **Penanganan Galat:**
 - ◊ Server tidak terjangkau/*timeout* → 504 Gateway Timeout.
 - ◊ Server mengembalikan galat → 502 Bad Gateway.
- Mencatat log: IP *client*, URL, status *cache* (HIT/MISS), dan waktu respons.
- **Konkurensi:** Wajib menangani beberapa *client* secara simultan.

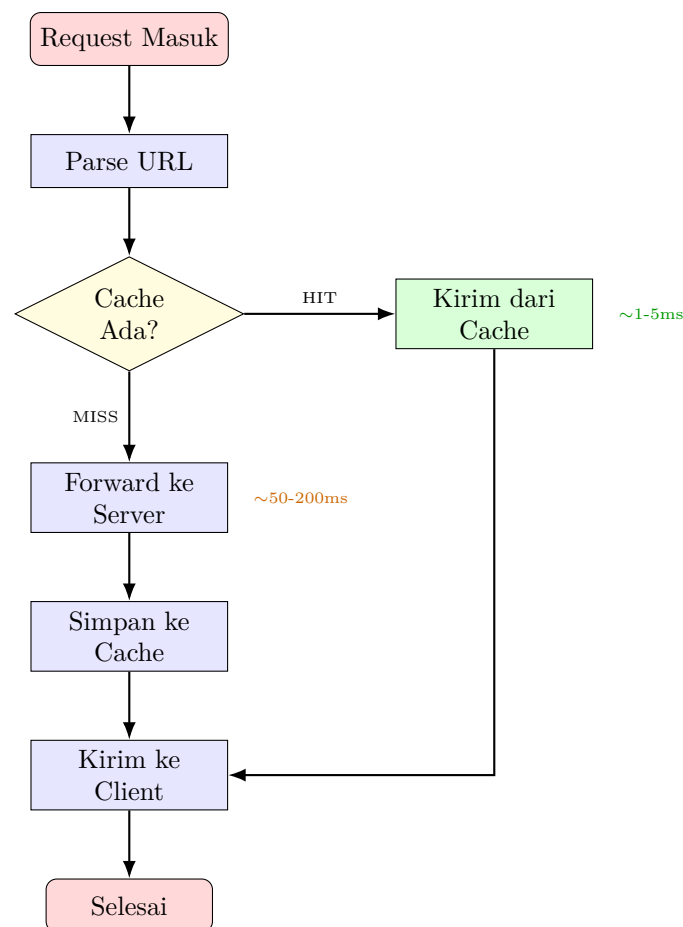


Figure 4: Alur logika caching pada Proxy Server

c. Client (`client.py`)

- **Mode HTTP (TCP):**
 - ◊ Mengirim *request* GET ke Proxy (`localhost:8080` atau alamat IP Proxy).
 - ◊ Menerima dan menampilkan *response* HTML pada terminal/*console*.
- **Mode QoS (UDP):**
 - ◊ Mengirim minimal 10 paket UDP ke port 9000 pada server.
 - ◊ Format *payload*: "Ping <seq> <timestamp>".

- *Timeout* per paket: maksimal 1 detik.
- Keluaran per paket: RTT (ms) atau "Request timed out".
- Statistik akhir: Min/Avg/Max RTT, *Packet Loss* (%), dan *Jitter*.

3. Integrasi Sistem (Wajib)

Topologi Komunikasi:

Client $\xrightarrow{\text{TCP/UDP}}$ Proxy $\xrightarrow{\text{TCP/UDP}}$ Web Server

Verifikasi Kepatuhan:

- ▶ **Log Server:** Hanya mencatat koneksi dari IP Proxy, *bukan client*.
- ▶ **Analisis Wireshark:** Tidak ada aliran HTTP langsung antara *client* dan *Web Server*.
- ▶ **Firewall/Binding:** *Web Server* tidak dapat diakses langsung melalui antarmuka jaringan *client*.

Pelanggaran ketentuan ini mengakibatkan invalidasi hasil pengujian.

4. Konkurensi dan Pengujian

- ▶ **Multithreading:** *Web Server* dan *Proxy* wajib menangani beberapa *client* secara simultan menggunakan `threading.Thread` atau model *thread-per-connection*.
- ▶ **Pengujian Multi-client:** Dokumentasikan pengujian dengan minimal **5 client** konkuren yang mengakses sistem secara bersamaan.
- ▶ **Penanganan Galat:** Program harus bersifat *graceful* (tidak *crash*) terhadap:
 - Koneksi *timeout* atau *reset*
 - Berkas tidak ditemukan (404)
 - *Request* HTTP yang tidak valid (*malformed*)

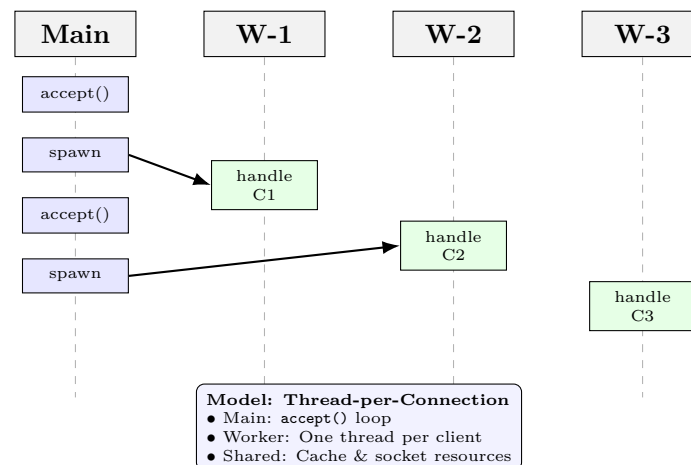


Figure 5: Model konkurensi: Main thread menerima koneksi, worker thread menangani request

F. Skenario Pengujian dan Analisis QoS

Bagian ini menjelaskan skenario pengujian **wajib** untuk memverifikasi fungsionalitas sistem dan mengukur performa jaringan berdasarkan parameter *Quality of Service* (QoS).

1. Tujuan Pengujian

- ▶ Memvalidasi fungsionalitas masing-masing komponen: *Client*, *Proxy*, dan *Web Server*.
- ▶ Memastikan integrasi sistem berjalan sesuai topologi *Client* → *Proxy* → *Server*.
- ▶ Menguji mekanisme *caching* (*HIT/MISS*) dan penanganan galat pada *Proxy*.
- ▶ Menguji kemampuan **multithreading** dalam menangani koneksi konkuren.
- ▶ Mengukur parameter QoS: *Throughput*, *Latency*, *Packet Loss*, dan *Jitter*.

2. Persiapan Pengujian

Daftar Periksa Pra-Pengujian:

- [1] Seluruh perangkat terhubung dalam satu jaringan lokal (LAN/WiFi) dengan konektivitas stabil.
- [2] Konfigurasi IP dan port konsisten:

Web Server: 8000 (TCP) / 9000 (UDP) · Proxy: 8080 · Client:
ephemeral
- [3] Berkas HTML/resource ditempatkan pada direktori yang sama dengan `webserver.py`.
- [4] Wireshark siap dijalankan pada antarmuka jaringan yang sesuai.
- [5] Sinkronisasi waktu antarperangkat (manual/NTP) untuk akurasi pengukuran RTT.

3. Skenario Pengujian Komponen

a. Web Server (`webserver.py`)

No	Langkah	Input/Eksekusi	Output yang Diharapkan
1	Menjalankan server	<code>python webserver.py</code>	Log: "Server running on port 8000/9000", <i>thread pool</i> siap
2	Permintaan berkas valid	GET <code>/index.html</code> via Proxy	Response 200 OK + konten HTML, log mencatat IP Proxy
3	Permintaan berkas tidak ditemukan	GET <code>/missing.html</code> via Proxy	Response 404 Not Found, log mencatat galat
4	Pengujian UDP echo	Mengirim paket UDP ke port 9000	Server memantulkan <i>payload</i> identik (<i>echo</i>)
5	Beban konkuren	5 <i>client</i> melakukan permintaan simultan	Semua permintaan ditangani, log menunjukkan pembuatan thread untuk tiap koneksi

b. Proxy Server (proxy.py)

No	Langkah	Input/Eksekusi	Output yang Diharapkan
1	Menjalankan proxy	<code>python proxy.py</code>	Log: "Proxy listening on port 8080", multi-threading aktif
2	Pengujian <i>forwarding</i>	<i>Client</i> meminta <code>/index.html</code>	Proxy meneruskan ke server, response diterima <i>client</i>
3	Cache MISS	Permintaan pertama <code>/page.html</code>	Proxy meneruskan ke server, menyimpan ke cache, log: MISS
4	Cache HIT	Permintaan kedua <code>/page.html</code> (sama)	Proxy mengirim dari cache lokal, log: HIT, respons lebih cepat
5	Propagasi galat	Server dimatikan, <i>client</i> meminta	Proxy mengembalikan 502 Bad Gateway atau 504 Gateway Timeout
6	Proxy konkuren	5 <i>client</i> via proxy secara simultan	Proxy menangani semua koneksi, cache tetap konsisten (tanpa <i>race condition</i>)

c. Client (client.py)

No	Langkah	Input/Eksekusi	Output yang Diharapkan
1	Mode TCP	<code>python client.py -mode tcp</code>	<i>Client</i> mengirim permintaan ke proxy, menampilkan response HTML di terminal
2	Mode UDP (QoS)	<code>python client.py -mode udp</code>	Output: RTT per paket, pesan timeout, statistik akhir (min/avg/max, loss%)
3	Validasi jalur	Memeriksa log server	Tidak ada log koneksi langsung dari IP <i>Client</i> ke Server
4	Verifikasi browser	Membuka <code>http://<proxy>:8080/</code> di browser	Halaman web tampil normal (opsional, validasi kepatuhan HTTP)
5	Simulasi multi-client	Menjalankan 5 instance <code>client.py</code>	Semua instance menerima response, tanpa <i>blocking/crash</i>

4. Skenario Multi-Client Konkuren (Wajib)

Tujuan: Menguji kemampuan sistem menangani beban simultan dan mengobservasi perilaku multithreading.

Prosedur:

- [1] Pastikan *Web Server* dan *Proxy* berjalan dengan multithreading aktif.
- [2] Jalankan **minimal 5 instance client.py** secara bersamaan menggunakan salah satu metode:
 - *Manual*: 5 terminal terpisah, eksekusi hampir bersamaan
 - *Script*: Python launcher dengan `subprocess` atau `threading`
 - *Distributed*: 5 perangkat berbeda dalam jaringan yang sama
- [3] Setiap *client* melakukan permintaan HTTP ke resource berbeda (uji cache MISS) dan sama (uji cache HIT).
- [4] Catat: waktu respons per *client*, log pembuatan thread, dan konsistensi cache.

Observasi Wajib:

- **Perilaku thread**: Log menunjukkan pembuatan thread baru untuk tiap koneksi masuk.
- **Konsistensi cache**: Tidak terjadi korupsi data atau *race condition* saat akses cache bersamaan.
- **Skalabilitas performa**: Waktu respons tidak meningkat drastis ($>2\times$) dibanding skenario satu *client*.
- **Penggunaan resource**: CPU/memori proxy dan server tetap stabil (tidak terjadi *memory leak*).

5. Pengukuran dan Analisis Quality of Service

Parameter QoS diukur melalui modul UDP pada `client.py` dengan ketentuan:

- ▶ Minimal 10 paket UDP dikirim ke port 9000 pada server.
- ▶ Timeout per paket: 1 detik.
- ▶ Format payload: "Ping <seq> <timestamp>".

Statistik Output Wajib:

Parameter	Definisi & Rumus
RTT (min/avg/max)	Waktu tempuh paket: $T_{recv} - T_{send}$
Packet Loss (%)	$\frac{\text{Paket tidak di-echo}}{\text{Total dikirim}} \times 100\%$
Jitter (ms)	Deviasi standar selisih RTT berturut-turut: $\sigma(RTT_i - RTT_{i-1})$
Throughput (kbps)	$\frac{\text{Total payload berhasil}}{\text{Durasi pengujian}}$

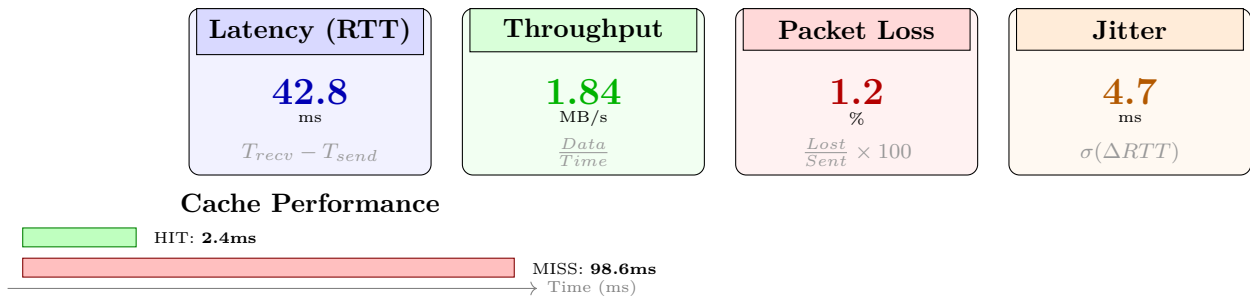


Figure 6: Dashboard metrik QoS dan perbandingan latency cache HIT vs MISS

Analisis Komparatif:

- Bandingkan RTT skenario satu *client* versus multi-*client* untuk mengobservasi dampak konkurensi.
- Analisis perbedaan latency cache HIT versus MISS sebagai bukti efektivitas *caching*.
- Evaluasi korelasi antara *packet loss* dan *jitter* pada kondisi jaringan berbeda (WiFi versus LAN kabel).

6. Analisis dengan Wireshark

- ▶ **Konfigurasi capture:** Jalankan Wireshark pada antarmuka jaringan yang digunakan, filter: `tcp.port==8000 || tcp.port==8080 || udp.port==9000`.
- ▶ **Analisis HTTP:** Verifikasi struktur request/response (method, header, status code, content-length).
- ▶ **Aliran TCP:** Amati *three-way handshake*, transfer data, dan terminasi koneksi untuk tiap sesi.
- ▶ **QoS UDP:** Validasi format payload, timestamp, dan urutan paket.
- ▶ **Aliran Konkuren:** Gunakan *Statistics* → *Conversations* untuk mengidentifikasi beberapa aliran TCP simultan.
- ▶ **Pemeriksaan Retransmisi:** Pada beban tinggi, amati apakah terjadi retransmisi TCP (indikasi kongesti).

7. Output yang Wajib Dikumpulkan

A. Dokumentasi Eksekusi (Tangkapan Layar):

- *Web Server*: log startup, penanganan permintaan, pembuatan thread, UDP echo.
- *Proxy Server*: log forwarding, cache HIT/MISS, penanganan galat (502/504).
- *Client*: output mode TCP (render HTML) dan mode UDP (statistik QoS).
- **Multi-client**: 5 instance berjalan simultan dengan response berhasil.
- Browser: halaman web tampil via proxy (opsional, validasi tambahan).
- Wireshark: capture aliran HTTP, TCP handshake, dan paket UDP.

B. Data Pengukuran QoS:

- Tabel RTT (min/avg/max), packet loss, jitter untuk 3 skenario: idle, beban sedang, beban tinggi.
- Grafik perbandingan latency cache HIT versus MISS.
- Analisis singkat: faktor yang memengaruhi variasi QoS dalam pengujian.

C. Analisis Multithreading:

- Bukti log: keluaran berselang (*interleaved*) dari beberapa thread.
- Observasi: apakah terjadi race condition pada cache? Bagaimana penanganannya?
- Estimasi: throughput maksimal sebelum degradasi performa signifikan.

8. Panduan Troubleshooting Umum

Gejala	Penyebab Umum	Solusi
Connection refused	Server/proxy belum berjalan atau port salah	Pastikan urutan start: Server → Proxy → Client; periksa konfigurasi port
Timeout pada UDP	Firewall memblokir UDP / server crash	Nonaktifkan firewall sementara; pastikan UDP server berjalan di port 9000
Cache tidak berfungsi	Jalur cache salah / izin akses ditolak	Periksa direktori cache relatif terhadap <code>proxy.py</code> ; pastikan dapat ditulis
Response HTML kosong	Parsing header HTTP salah	Validasi format request: <code>GET /path HTTP/1.1\r\n\r\n</code> (double CRLF)
Multi-client blocking	Thread tidak dimulai / <code>join()</code> prematur	Pastikan <code>thread.start()</code> dipanggil, hindari <code>join()</code> dalam loop accept
Wireshark tidak capture	Antarmuka / filter salah	Pilih antarmuka aktif; gunakan filter <code>ip.addr==<target_IP></code>

G. Laporan Akhir

Laporan akhir disusun dalam format PDF dan wajib menjelaskan seluruh proses implementasi, integrasi sistem, serta hasil pengujian.

Struktur Laporan:

1. Cover

- Judul tugas besar
- Nama dan NIM anggota kelompok
- Nama mata kuliah
- Nama dosen pengampu
- Program studi dan universitas
- Tanggal pengumpulan

2. Pembagian Tugas

- Deskripsi kontribusi masing-masing anggota dalam implementasi dan pengujian.

3. Implementasi Sistem

- Penjelasan arsitektur dan alur komunikasi *Client-Proxy-Server*.
- Dokumentasi kode inti: mekanisme *socket*, *forwarding*, *caching*, dan UDP *echo*.
- Justifikasi desain: penanganan konkurensi, manajemen *thread*, dan strategi *error handling*.

4. Analisis QoS dan Multithreading

- **Perhitungan QoS:** Penyajian data RTT (min/avg/max), *packet loss*, *jitter*, dan *throughput*.
- **Analisis Faktor:** Identifikasi variabel yang memengaruhi performa jaringan (kondisi medium, beban, konfigurasi).
- **Studi Komparatif:** Perbandingan latency *cache HIT* vs *MISS*, serta skenario satu *client* vs multi-*client*.
- **Evaluasi Multithreading:** Analisis skalabilitas sistem dan efektivitas model *thread-per-connection*.

5. Troubleshooting (Penanganan Kendala)

- Dokumentasikan kendala teknis selama implementasi dan pengujian beserta solusi yang diterapkan.
- Disarankan menggunakan format tabel berikut:

Table 5: Contoh Format Tabel Troubleshooting

Masalah	Penyebab	Solusi
Koneksi ditolak	Port salah / server belum berjalan	Verifikasi konfigurasi port dan urutan inisialisasi
Berkas tidak tampil	Jalur berkas (<i>path</i>) tidak valid	Perbaiki referensi jalur berkas pada <i>web server</i>
<i>Timeout</i> UDP	Paket hilang / latensi tinggi	Implementasi mekanisme <i>timeout handling</i> dan retransmisi
Korupsi data cache	<i>Race condition</i> pada penulisan	Implementasi <i>file locking</i> atau penulisan atomik
<i>Thread leak</i>	Siklus hidup thread tidak terminasi	Manajemen <i>lifecycle</i> thread yang tepat (<i>join/daemon</i>)

- Setiap kendala wajib disertai analisis singkat mengenai akar masalah dan dampak solusi.

6. Kesimpulan dan Saran

- Rangkuman capaian pembelajaran dan keterbatasan implementasi.
- Rekomendasi pengembangan untuk skalabilitas dan fitur tambahan.

7. Referensi

- Cantumkan sumber pustaka, dokumentasi resmi, dan referensi teknis yang digunakan.

H. Slide Presentasi

Mahasiswa wajib menyiapkan materi presentasi dalam format PDF dengan durasi penyajian yang telah ditentukan.

Struktur Minimal Slide:

- **Slide 1: Cover**
 - Judul tugas besar
 - Nama anggota kelompok dan NIM
 - Program studi dan universitas
- **Slide 2: Latar Belakang dan Arsitektur**
- **Slide 3–4: Implementasi**
- **Slide 5–6: Demo Sistem**
- **Slide 7–8: Hasil Pengujian dan Analisis QoS**
- **Slide 9: Analisis Multithreading dan Penanganan Konkuren**
- **Slide Terakhir: Kesimpulan**